

Laboratorio Python + Analisi Dati

Manuale Operativo e Guida Pratica ai Laboratori

■ ■ INFORMAZIONI GENERALI SUL CORSO

Corso: Laboratorio Python + Analisi Dati (22 Ore)
Docente: Arnaldo Morena
Istituzione: ITIS Campobasso
Tipologia: Manuale Didattico Operativo & Guida ai Laboratori
Prerequisiti: Fondamenti minimi di logica e utilizzo base del PC
Struttura: 8 Capitoli Operativi + Project Work Finale + Appendice di Riferimento

Questa dispensa fornisce il percorso completo ed autosufficiente per consentire a ogni studente di seguire le lezioni in aula, riprodurre autonomamente tutti i laboratori pratici, comprendere i costrutti architetturali e completare con successo il Project Work finale.

Introduzione al Corso e Setup dell'Ambiente

Obiettivi Formativi del Corso

Il corso 'Laboratorio Python + Analisi Dati' è un percorso intensivo e pratico di 22 ore ideato per fornire competenze solide, moderne e direttamente spendibili nel mondo aziendale per l'analisi, la pulizia, l'automazione e la visualizzazione interattiva dei dati.

A differenza dei corsi puramente teorici, questo percorso adotta una metodologia hands-on basata su un unico caso aziendale continuo: la catena retail 'TechStore Italia'. Gli studenti affronteranno problematiche reali di ingestione dati da fogli Excel distribuiti su più filiali territoriali (Roma, Milano, Torino), pulizia di anomalie e formati disomogenei, consolidamento in pipeline scalabili, creazione di report grafici professionali e pubblicazione di una dashboard interattiva su server Linux.

- **Praticità immediata:** Ogni nozione teorica è seguita da notebook operativi ed esercizi pratici.
- **Integrazione end-to-end:** Dall'esplorazione dei tipi Python nativi fino al deploy di servizi systemd in ambiente Linux server.
- **Standard industriali:** Utilizzo delle librerie de facto standard dell'ecosistema scientifico Python: Pandas, NumPy, OpenPyXL, Matplotlib, Seaborn, PyArrow e Streamlit.

Architettura Didattica e Flusso Operativo

L'architettura del corso guida lo studente lungo una progressione naturale: dai fondamenti algoritmici alla gestione di pipeline automatizzate e dashboard per il management.

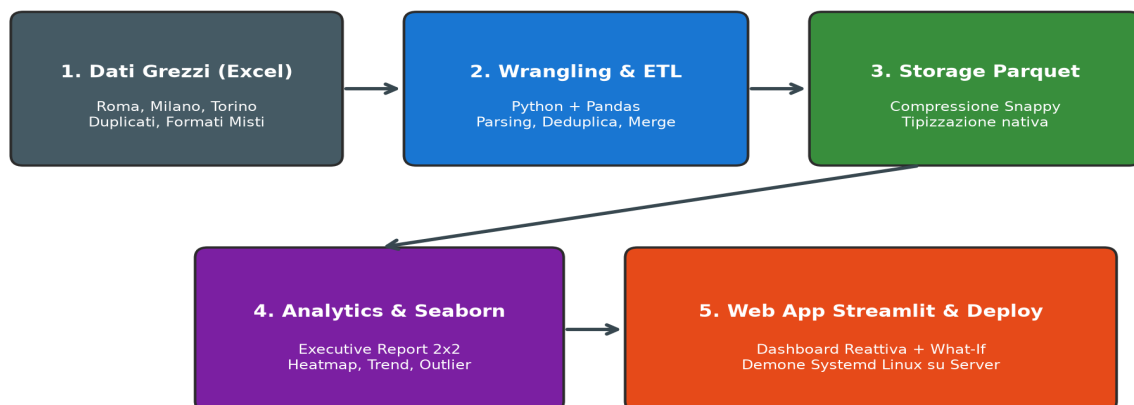


Figura: Architettura modulare e flusso di avanzamento didattico (22 Ore)

Modulo	Ore	Argomento Principale	Deliverable Chiave
Modulo 1	2h	Python Operativo & Strutture Dati	Script di calcolo aggregato su List[Dict]
Modulo 2	4h	Pandas Fondamentale	Esplorazione, filtri .loc/.iloc, gestione copie
Modulo 3	3h	Data Wrangling & Qualità Dati	Funzione di pulizia e merge relazionale validato
Modulo 4	3h	Visualizzazione & Reporting	Dashboard grafica 2x2 ad alta risoluzione (300 DPI)
Modulo 5	2h	Automazione Pipeline ETL	Pipeline batch, storage Parquet, report Excel
Modulo 6	4h	Dashboard Interattiva Streamlit	Web app reattiva multi-tab con What-If simulator
Modulo 7	1h	Deploy Linux & Systemd	Demone di produzione persistente su porta 8501
Project Work	3h	Integrazione Filiale Napoli	Audit, pipeline, dashboard e reportistica finale

Installazione e Configurazione dell'Ambiente di Sviluppo

Per garantire la riproducibilità totale degli esperimenti e prevenire conflitti tra versioni di librerie, è tassativo operare all'interno di un ambiente virtuale (virtualenv) dedicato con Python 3.10 o superiore.

■ Setup ambiente virtuale e installazione dipendenze (Bash / Terminale)

```
# 1. Clonare il repository del corso
git clone https://github.com/arnymore/python_analisi_dati_campobasso.git
cd python_analisi_dati_campobasso

# 2. Creare l'ambiente virtuale isolato
python3 -m venv .venv

# 3. Attivare l'ambiente virtuale
source .venv/bin/activate      # Su Linux / macOS
# .venv\Scripts\activate      # Su Windows (PowerShell/CMD)

# 4. Aggiornare pip e installare i package richiesti
pip install --upgrade pip
pip install -r requirements.txt

# 5. Avviare Jupyter Lab per i laboratori
jupyter lab
```

■■ VERIFICA DELL'INSTALLAZIONE

Per verificare la corretta configurazione, eseguire da terminale: `python -c 'import pandas, streamlit, matplotlib, seaborn, pyarrow; print("Ambiente pronto!")'`. Se il comando stampa il messaggio senza errori, l'ambiente è perfettamente operativo.

Struttura del Repository di Lavoro

Il repository del corso è organizzato in modo modulare per rispecchiare fedelmente l'organizzazione di un reale progetto software enterprise:

- **dataset/raw/**: Contiene i file Excel sorgente originali (roma.xlsx, milano.xlsx, torino.xlsx, napoli_project_work.xlsx).
- **dataset/generated/**: Cartella di output per i dataset processati (dataset_master.parquet, report_consolidato.xlsx).
- **laboratori/**: Jupyter Notebook guidati con esercizi a completamento per ciascun modulo.
- **soluzioni/**: Notebook risolti e commentati dal docente con soluzioni di riferimento certificate.
- **dashboard/**: Codice sorgente della web application Streamlit (app.py) e moduli ausiliari.
- **scripts/**: Script Python di supporto per la generazione dataset e build della documentazione.
- **project_work/**: Specifiche, benchmark e checklist per la prova pratica finale.

Capitolo 1 - Python Operativo per l'Analisi Dati

■ ■ SCHEDA MODULO DIDATTICO[illegible]

laboratori/lab_01_python_operativo.ipynb

Obiettivo Didattico: Strutture dati native, logica funzionale e costrutti algoritmici essenziali

1.1 Tipi di Dato Primitivi e Collezioni Native

Python offre un sistema di tipi dinamico ma fortemente tipizzato. Nelle applicazioni di elaborazione dati, comprendere a fondo il comportamento in memoria dei tipi primitivi (int, float, str, bool) e delle collezioni native è indispensabile prima di passare a strutture ad alte prestazioni come quelle di Pandas.

Le due strutture dati cardine in Python sono le liste (sequenze ordinate e mutabili) e i dizionari (tabelle hash composte da coppie chiave-valore). La combinazione di queste due strutture - la cosiddetta lista di dizionari ('List of Dicts') - rappresenta il ponte naturale tra il codice Python puro e le tabelle relazionali o i DataFrame.

■ Rappresentazione tabellare tramite List[Dict] nativa

```
# Modellazione delle transazioni di vendita con strutture dati native
transazioni = [
    {'id_transazione': 'T001', 'cliente': 'Acme Corp', 'prodotto': 'Notebook Pro', 'qta': 3, 'prezzo': 120
0.0},
    {'id_transazione': 'T002', 'cliente': 'Beta SpA', 'prodotto': 'Monitor 27"', 'qta': 5, 'prezzo': 350.0
},
    {'id_transazione': 'T003', 'cliente': 'Acme Corp', 'prodotto': 'Mouse Wireless', 'qta': 10, 'prezzo':
25.0},
    {'id_transazione': 'T004', 'cliente': 'Gamma Srl', 'prodotto': 'Notebook Pro', 'qta': 1, 'prezzo': 120
0.0}
]

# Calcolo del fatturato totale per cliente mediante dizionario di accumulo
fatturato_per_cliente = {}
for t in transazioni:
    importo = t['qta'] * t['prezzo']
    cliente = t['cliente']
    fatturato_per_cliente[cliente] = fatturato_per_cliente.get(cliente, 0.0) + importo

for cliente, totale in fatturato_per_cliente.items():
    print(f'Cliente: {cliente:<12} | Totale Fatturato: € {totale:,.2f}')
```

1.2 List Comprehension e Funzioni di Elaborazione

Le 'List Comprehensions' e le 'Dict Comprehensions' offrono una sintassi concisa, leggibile e computazionalmente più efficiente rispetto ai cicli 'for' tradizionali per trasformare e filtrare collezioni di dati.

La scrittura di funzioni pure e modulari consente di incapsulare regole di business (come calcoli di sconti scaglionati o normalizzazioni di stringhe) garantendo manutenibilità e testabilità del codice.

■ Filtri con List Comprehension e calcolo sconti avanzati

```
def calcola_sconto_commerciale(importo: float, categoria: str) -> float:
    '''Applica sconti differenziati in base al volume e categoria.'''
    if importo > 3000.0 or categoria == 'Hardware':
        return importo * 0.10 # 10% di sconto
    elif importo > 1000.0:
        return importo * 0.05 # 5% di sconto
    return 0.0

# Estrazione ordini di alto valore (> 1000€) con calcolo importo netto
ordini_premium = [
    {
        'id': t['id_transazione'],
        'cliente': t['cliente'],
        'lordo': t['qta'] * t['prezzo'],
        'sconto': calcola_sconto_commerciale(t['qta'] * t['prezzo'], 'Hardware'),
        'netto': (t['qta'] * t['prezzo']) - calcola_sconto_commerciale(t['qta'] * t['prezzo'], 'Hardware')
    }
    for t in transazioni
    if (t['qta'] * t['prezzo']) >= 1000.0
]

print('Transazioni Premium filtrate:', len(ordini_premium))
```

■ ATTENZIONE ALLA MUTABILITÀ DEGLI OGGETTI

In Python le liste e i dizionari sono passati per riferimento. Assegnare `lista_b = lista_a` non crea una copia ma crea un secondo puntatore alla medesima area di memoria. Per creare una copia indipendente di strutture annidate, utilizzare sempre `import copy; lista_b = copy.deepcopy(lista_a)`.

1.3 Errori Comuni e Best Practice

- **KeyError su dizionari non controllati:** Evitare l'accesso diretto `d['chiave']` quando la chiave potrebbe non esistere; preferire sempre il metodo `d.get('chiave', default)`.
- **Division by Zero:** Nei calcoli di indicatori percentuali o prezzi unitari, gestire esplicitamente i denominatori nulli o uguali a zero con controlli `if` preventivi o blocchi `try/except`.
- **Manipolazione di liste durante l'iterazione:** Non rimuovere né aggiungere elementi da una lista mentre la si sta scorrendo in un ciclo `for`. Iterare invece su una copia o utilizzare una comprehension.

1.4 Domande di Autovalutazione

■ Domanda 1: Qual è la differenza fondamentale tra una lista (list) e una tupla (tuple) in Python?

■ **Risposta:** La lista è una sequenza mutabile (può essere modificata in-place dopo la creazione), mentre la tupla è immutabile (non può essere alterata, garantendo integrità dei dati e potendo fungere da chiave nei dizionari).

■ Domanda 2: Cosa restituisce il metodo `d.get('totale', 0.0)` se la chiave 'totale' non è presente nel dizionario `d`?

■ **Risposta:** Restituisce il valore di fallback `0.0` senza sollevare l'eccezione `KeyError`, prevenendo il crash dello script.

■ Domanda 3: Perché la struttura `List[Dict]` è considerata il predecessore concettuale di un `DataFrame` Pandas?

■ **Risposta:** Perché ogni dizionario della lista rappresenta un record (riga), con le relative coppie chiave-valore che definiscono i nomi delle colonne e i valori associati.

Capitolo 2 - Pandas Fondamentale per l'Analisi Tabellare

■ ■ SCHEDA MODULO DIDATTICO

Durata del Modulo: 4 Ore | **Laboratorio:**

laboratori/lab_02_pandas_fondamentale.ipynb
Obiettivo Didattico: DataFrame, Series, indicizzazione, filtri booleani e gestione delle copie in memoria

2.1 L'Architettura di Pandas: Series e DataFrame

Pandas è la libreria open source cardine per l'analisi e la manipolazione di dati strutturati in Python. Il suo funzionamento si basa su due strutture dati fondamentali:

1. Series: array monodimensionale etichettato, capace di contenere qualsiasi tipo di dato. È dotata di un 'Index' che associa un'etichetta a ciascun elemento.
2. DataFrame: struttura dati bidimensionale tabellare con colonne etichettate (ciascuna delle quali è una Series) e un indice comune di riga.

L'ingestione di file Excel in Pandas avviene tramite la funzione 'pd.read_excel()', che sfrutta sotto il cofano l'engine OpenPyXL per convertire i fogli di calcolo in DataFrame ad alte prestazioni.

■ Importazione Excel, ispezione metadati e tipi di dato

```
import pandas as pd

# Caricamento del dataset vendite della filiale di Roma
df_roma = pd.read_excel('dataset/raw/roma.xlsx')

# Ispezione delle dimensioni e dei tipi colonna
print(f'Dimensioni dataset Roma: {df_roma.shape[0]} righe x {df_roma.shape[1]} colonne')
print('--- Info Tipi di Dato ---')
print(df_roma.dtypes)
print('--- Prime 3 righe ---')
print(df_roma.head(3))
```

2.2 Indicizzazione e Selezione: .loc vs .iloc e Filtri Booleani

La selezione dei dati in Pandas deve avvenire rigorosamente tramite gli accessor standard per evitare ambiguità sintattiche e garantire performance:

- `.loc[]`: selezione basata su etichette (nomi di colonna ed indici espliciti).
- `.iloc[]`: selezione puramente posizionale basata su interi (0-indexed).

I filtri booleani consentono di estrarre sottoinsiemi di dati combinando condizioni logiche complesse mediante gli operatori vettoriali bitwise: & (AND), | (OR), ~ (NOT). Ciascuna condizione deve essere tassativamente racchiusa tra parentesi tonde.

■ Filtri booleani composti e selezione mirata con .loc

```
# Estrazione vendite di 'Notebook' con importo lordo > 2000€ e data 2024
maschera_notebook = (df_roma['prodotto'] == 'Notebook Pro') & (df_roma['prezzo_unitario'] * df_roma['quantita'] > 2000)

# Selezione mirata di colonne specifiche con .loc
df_selezionato = df_roma.loc[maschera_notebook, ['id_transazione', 'cliente', 'quantita', 'prezzo_unitario']]

print(f'Trovate {len(df_selezionato)} transazioni corrispondenti ai criteri.')
print(df_selezionato.head())
```

2.3 Il Problema Critico: View vs Copy e SettingWithCopyWarning

Uno degli errori più subdoli commessi dagli sviluppatori in Pandas è il 'SettingWithCopyWarning'. Quando si crea un sottoinsieme di un DataFrame tramite un filtro, Pandas può restituire una 'Vista' (View) dell'area di memoria originaria oppure una 'Copia' (Copy) indipendente.

Se si tenta di modificare una colonna su una vista senza aver creato una copia esplicita, l'assegnazione potrebbe fallire silenziosamente o sovrascrivere in modo imprevedibile il DataFrame originario. Per evitare qualsiasi anomalia, ogni operazione di filtraggio che prelude a una modifica deve terminare con '.copy()'.

■ Gestione corretta della copia per evitare SettingWithCopyWarning

```
# ■ APPROCCIO ERRATO (genera SettingWithCopyWarning):
# df_sub = df_roma[df_roma['quantita'] > 5]
# df_sub['fatturato_lordo'] = df_sub['quantita'] * df_sub['prezzo_unitario']

# ■ APPROCCIO CORRETTO E SICURO:
df_sub = df_roma.loc[df_roma['quantita'] > 5].copy()
df_sub['fatturato_lordo'] = df_sub['quantita'] * df_sub['prezzo_unitario']
df_sub['prezzo_scontato'] = df_sub['prezzo_unitario'] * (1 - df_sub['sconto_applicato'].fillna(0))

print('Colonna aggiunta in sicurezza. Righe nel sottoinsieme:', len(df_sub))
```

■ ■ REGOLA AUREA DI PANDAS

Ogni volta che create un sottoinsieme con un filtro o una slice e intendete modificare, aggiungere o sovrascrivere colonne, applicate sempre il metodo .copy(). Questo garantisce l'allocazione di un nuovo blocco di memoria indipendente.

2.4 Errori Comuni e Best Practice

- **Parentesi omesse nei filtri booleani:** Scrivere `df['qta'] > 5 & df['prezzo'] > 100` causa un errore di precedenza operatori. Scrivere sempre `(df['qta'] > 5) & (df['prezzo'] > 100)`.
- **Confusione tra .loc e .iloc:** Ricordare che `df.iloc[0:5]` seleziona le prime 5 righe (indici posizionali 0,1,2,3,4), mentre `df.loc[0:5]` include anche l'etichetta 5 se presente nell'indice.
- **Modifiche su viste senza .copy():** Causa primaria di corruzione silenziosa dei dati e warning in fase di esecuzione.

2.5 Domande di Autovalutazione

■ Domanda 1: Cosa differenzia una Series da un DataFrame in Pandas?

■ **Risposta:** Una Series è una struttura monodimensionale con un singolo array di valori e un indice; un DataFrame è una tabella bidimensionale composta da più Series che condividono lo stesso indice di riga.

■ Domanda 2: Come si elimina definitivamente il rischio di generare un SettingWithCopyWarning?

■ **Risposta:** Utilizzando .loc per l'assegnazione diretta oppure invocando esplicitamente il metodo .copy() al momento dell'estrazione del subset prima di effettuare modifiche.

■ Domanda 3: Quale parametro di pd.read_excel permette di specificare il nome o l'indice del foglio da caricare?

■ **Risposta:** Il parametro sheet_name (es. sheet_name='Vendite' o sheet_name=0 per il primo foglio).

Capitolo 3 - Data Wrangling e Qualità del Dato

■ ■ SCHEDA MODULO DIDATTICO[illegible]

3.1 La Gestione dei Duplicati e dei Missing Values (NaN)

Nei contesti aziendali reali, i dati grezzi estratti da sistemi legacy o fogli Excel manuali presentano inevitabilmente anomalie: righe duplicate accidentalmente, campi nulli e valori mancanti.

La bonifica dei record duplicati avviene tramite `'drop_duplicates()'`. Per quanto riguarda i valori mancanti, dopo averne quantificato l'incidenza con `'isna().sum()'`, è possibile procedere all'eliminazione mirata (`'dropna()'`) o all'imputazione strategica (`'fillna()'`). Nel percorso standard del corso, l'imputazione di grandezze numeriche (es. sconti o prezzi) avviene tramite la mediana globale del dataset, garantendo robustezza rispetto agli outlier.

■ Audit e bonifica duplicati e missing values

```
import pandas as pd
import numpy as np

df = pd.read_excel('dataset/raw/roma.xlsx')

# 1. Rilevazione ed eliminazione duplicati completi
n_dup = df.duplicated().sum()
print(f'Record duplicati rilevati: {n_dup}')
df_clean = df.drop_duplicates(subset=['id_transazione'], keep='first').copy()

# 2. Analisi dei valori nulli
print('Missing values per colonna prima della bonifica:')
print(df_clean.isna().sum())

# 3. Imputazione missing values: sconti nulli a 0, prezzo unitario con mediana globale
df_clean['sconto_applicato'] = df_clean['sconto_applicato'].fillna(0.0)
mediana_prezzo = df_clean['prezzo_unitario'].median()
df_clean['prezzo_unitario'] = df_clean['prezzo_unitario'].fillna(mediana_prezzo)

print(f'Imputazione completata. Mediana prezzo applicata: € {mediana_prezzo:.2f}')
```

3.2 Normalizzazione Robusta di Date Disomogenee

Uno dei problemi più frequenti e deleteri nell'importazione da fogli di calcolo è la disomogeneità dei formati data: alcune celle contengono stringhe in formato italiano 'DD/MM/YYYY', altre in formato anglosassone 'YYYY-MM-DD', timestamp o valori corrotti.

Per gestire questa casistica in modo sistematico, implementiamo la funzione `'parse_data_flessibile()'`, che applica un parsing robusto con `'pd.to_datetime(..., errors='coerce')'` e fallback su formati specifici, convertendo tutte le date in oggetti `'datetime64[ns]'` standard.

■ Funzione standard di normalizzazione temporale

```
def parse_data_flessibile(colonna_date: pd.Series) -> pd.Series:
    '''Converte una colonna eterogenea di date in datetime standard.'''
    # Tentativo primario con inferenza automatica e coercion dei valori invalidi
    date_parsed = pd.to_datetime(colonna_date, format='%Y-%m-%d', errors='coerce')

    # Fallback per il formato europeo giorno/mese/anno
    maschera_null = date_parsed.isna()
    if maschera_null.any():
        date_fallback = pd.to_datetime(colonna_date[maschera_null], format='%d/%m/%Y', errors='coerce')
        date_parsed.update(date_fallback)

    return date_parsed

df_clean['data_transazione'] = parse_data_flessibile(df_clean['data'])
df_clean['anno_mese'] = df_clean['data_transazione'].dt.to_period('M')
print('Date normalizzate con successo. Intervallo:', df_clean['data_transazione'].min(), '->', df_clean['data_transazione'].max())
```

3.3 Merge Relazionale con Validazione di Integrità

L'arricchimento del dataset vendite con anagrafiche esterne (es. anagrafica clienti o catalogo prodotti) richiede l'esecuzione di join relazionali tramite la funzione 'pd.merge()'.

Per prevenire la moltiplicazione incontrollata delle righe (esplosione cartesiana generata da chiavi duplicate nella tabella anagrafica), è fondamentale utilizzare il parametro 'validate='many_to_one' (m:1) e tracciare la provenienza dei record con 'indicator=True'.

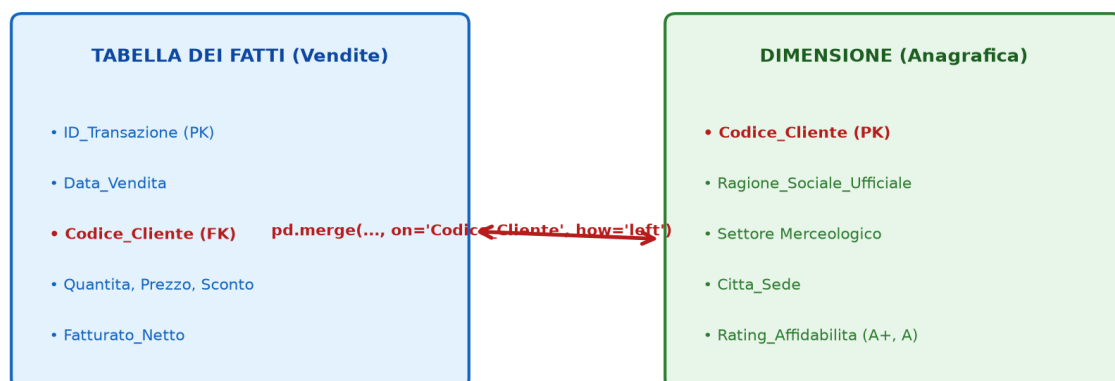


Figura: Architettura del Merge Relazionale con validazione 'many_to_one' e controllo integrità

■ Merge relazionale validato e asserzione di integrità

```
# Creazione anagrafica categorie e listino prodotti
df_catalogo = pd.DataFrame({
    'prodotto': ['Notebook Pro', 'Monitor 27"', 'Mouse Wireless', 'Tastiera Meccanica', 'Docking Station'],
    'categoria': ['Hardware', 'Periferiche', 'Accessori', 'Accessori', 'Hardware'],
    'costo_base': [800.0, 210.0, 12.0, 45.0, 95.0]
})

# Record pre-merge per verifica integrità
righe_prima = len(df_clean)

# Merge con validazione rigorosa Many-to-One
df_merged = pd.merge(
    df_clean,
    df_catalogo,
    on='prodotto',
    how='left',
    validate='many_to_one',
    indicator=True
)

# Asserzione di controllo qualità: il numero di righe non deve variare
assert len(df_merged) == righe_prima, 'ERRORE CRITICO: Rilevata esplosione righe durante il merge!'
print(f'Merge completato con successo su {len(df_merged)} record.')
print(df_merged['_merge'].value_counts())
```

■ BEST PRACTICE DI INGEGNERIA DEL DATO

L'uso di assert sui DataFrame dopo operazioni critiche (come merge o filtri) garantisce il fallimento immediato e controllato della pipeline qualora i dati sorgente violassero le ipotesi relazionali.

3.4 Approfondimento Avanzato: Imputazione Contestuale con transform()

Mentre nel percorso operativo standard l'imputazione avviene con mediana globale, in scenari avanzati è possibile imputare i valori mancanti calcolando statistiche raggruppate per categoria tramite 'groupby().transform()':

Esempio avanzato: `df['prezzo'] = df['prezzo'].fillna(df.groupby('categoria')['prezzo'].transform('median'))`

Questo approccio assegna a ciascun prodotto mancante la mediana specifica della sua categoria merceologica.

3.5 Errori Comuni e Best Practice

- **Esplosione cartesiana da merge senza validazione:** Se la tabella destra contiene chiavi duplicate, un merge left senza `validate='many_to_one'` duplica silenziosamente le righe delle transazioni falsificando il fatturato totale.
- **Parsing date con giorno e mese invertiti:** L'ambiguità tra '01/02/2024' (1 Febbraio o 2 Gennaio) va risolta specificando sempre il format esplicito in `pd.to_datetime`.
- **fillna() su colonne non numeriche:** Assicurarsi di imputare valori coerenti con il tipo di dato (es. stringhe vuote o 'Sconosciuto' per testi, 0 o mediane per numeri).

3.6 Domande di Autovalutazione

■ **Domanda 1:** Cosa accade se in `pd.merge()` si specifica `validate='many_to_one'` ma la tabella di destra contiene chiavi duplicate?

■ **Risposta:** Pandas solleva immediatamente un'eccezione `'MergeError'`, bloccando l'esecuzione ed evitando la corruzione silenziosa dei dati.

■ **Domanda 2:** Perché si preferisce l'imputazione tramite mediana rispetto alla media aritmetica in presenza di outlier?

■ **Risposta:** La mediana è un indicatore di posizione robusto, insensibile a valori estremi o errati (outlier), mentre la media viene pesantemente distorta.

■ Domanda 3: A cosa serve il parametro indicator=True nella funzione merge?

■ **Risposta:** Aggiunge al DataFrame risultante una colonna speciale '_merge' con valori 'both', 'left_only' o 'right_only', permettendo di verificare quali record hanno trovato corrispondenza.

Capitolo 4 - Visualizzazione dei Dati e Reporting Esecutivo

■ ■ SCHEDA MODULO DIDATTICO[illegible]

laboratori/lab_04_visualizzazione_reporting.ipynb
Obiettivo Didattico: Architettura OOP di Matplotlib, Seaborn per statistiche avanzate e composizione di dashboard 2x2 ad alta risoluzione

4.1 L'Architettura Object-Oriented di Matplotlib

In ambito professionale, l'utilizzo di Matplotlib non deve basarsi sull'interfaccia procedurale 'pyplot' (stile MATLAB), bensì sull'architettura Object-Oriented (OO).

Il pattern standard prevede l'istanziamento esplicita degli oggetti 'Figure' (la tela complessiva) e 'Axes' (il singolo sistema di coordinate cartesiane su cui tracciare grafici, etichette e assi):

```
'fig, ax = plt.subplots(figsize=(10, 6))'
```

Questo approccio garantisce il controllo millimetrico su ogni elemento grafico, la riutilizzabilità del codice e la composizione modulare di layout multi-pannello.

4.2 Grafici Statistici con Seaborn

Seaborn si interfaccia nativamente con i DataFrame di Pandas e offre pattern grafici di alto livello ideali per l'esplorazione aziendale:

- Lineplot: analisi delle serie storiche e trend temporali di vendita con aggregazione automatica.
- Boxplot: distribuzione statistica di prezzi, sconti o marginalità per categoria, evidenziando quartili ed anomalie (outlier).
- Heatmap: matrice di correlazione tra indicatori numerici o mappe di intensità incrociata (es. Mese vs Categoria).

■ Creazione di grafici statistici con Seaborn e Matplotlib OOP

```
import matplotlib.pyplot as plt
import seaborn as sns

# Configurazione stile globale
sns.set_theme(style='whitegrid', palette='Blues_r')

# 1. Analisi Trend Mensile del Fatturato
fig, ax = plt.subplots(figsize=(10, 4.5))
sns.lineplot(data=df_merged, x='mese', y='fatturato_netto', estimator='sum', errorbar=None, marker='o', ax=ax, color='#1E88E5', lw=2.5)
ax.set_title('Evoluzione Mensile Fatturato Netto Totale (2024)', fontsize=13, fontweight='bold', pad=12)
ax.set_xlabel('Mese di Vendita', fontsize=11)
ax.set_ylabel('Fatturato Netto (€)', fontsize=11)
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda x, loc: f'€ {x*1e-3:.0f}k'))
plt.tight_layout()
plt.close(fig)
```

4.3 Composizione dell'Executive Dashboard 2x2 ed Export a 300 DPI

Per produrre report grafici destinati al management esecutivo, componiamo un layout a griglia 2x2 su una singola figura ad alta risoluzione (300 DPI), esportata in formato vettoriale o PNG per l'inserimento in presentazioni e relazioni ufficiali.

■ Generazione completa della Dashboard 2x2 e salvataggio a 300 DPI

```
# Creazione griglia 2x2 con dimensioni proporzionate per stampa A4 orizzontale
fig, axes = plt.subplots(nrows=2, ncols=2, figsize=(14, 9))
fig.suptitle('EXECUTIVE SALES REPORT 2024 - TECHSTORE ITALIA', fontsize=16, fontweight='bold', y=0.98, color='#0F2942')

# Panel 1 (Top-Left): Trend Mensile
sns.lineplot(data=df_merged, x='mese', y='fatturato_netto', estimator='sum', errorbar=None, ax=axes[0, 0],
             color='#1E88E5', marker='s')
axes[0, 0].set_title('Trend Fatturato Mensile', fontweight='bold')

# Panel 2 (Top-Right): Fatturato per Categoria Prodotto
cat_sales = df_merged.groupby('categoria')['fatturato_netto'].sum().sort_values(ascending=False)
sns.barplot(x=cat_sales.values, y=cat_sales.index, ax=axes[0, 1], palette='Blues_r')
axes[0, 1].set_title('Fatturato Totale per Categoria', fontweight='bold')

# Panel 3 (Bottom-Left): Distribuzione Sconti per Categoria (Boxplot)
sns.boxplot(data=df_merged, x='categoria', y='sconto_applicato', ax=axes[1, 0], palette='Set2')
axes[1, 0].set_title('Politica Sconti: Distribuzione per Categoria', fontweight='bold')

# Panel 4 (Bottom-Right): Top 5 Clienti per Fatturato
top_clienti = df_merged.groupby('cliente')['fatturato_netto'].sum().nlargest(5)
sns.barplot(x=top_clienti.values, y=top_clienti.index, ax=axes[1, 1], palette='crest')
axes[1, 1].set_title('Top 5 Clienti Corporate', fontweight='bold')

# Formattazione globale e salvataggio a 300 DPI
plt.tight_layout()
fig.savefig('slides/assets/dashboard_executive.png', dpi=300, bbox_inches='tight')
plt.close(fig)
print('Executive Dashboard esportata con successo a 300 DPI.')
```

■ ESPORTAZIONE DI REPORT PROFESSIONALI

Il parametro 'dpi=300' assicura una definizione di livello editoriale (stampa su carta o visualizzazione su schermi 4K), mentre 'bbox_inches="tight"' rimuove automaticamente margini bianchi e testi tagliati ai bordi.

4.4 Errori Comuni e Best Practice

- **Miscelare sintassi plt.* con ax.*:** Genera comportamenti imprevedibili nel targeting dei sotto-grafici. Inizializzare sempre fig, ax e richiamare i metodi sull'oggetto ax (es. ax.set_title, ax.set_xlabel).
- **Etichette degli assi illeggibili o sovrapposte:** Utilizzare plt.xticks(rotation=45) o ax.tick_params(axis='x', rotation=45) e invocare sempre plt.tight_layout().
- **Mancata formattazione degli importi monetari:** Evitare numeri grezzi come '1500000.00'; utilizzare formattatori personalizzati per esprimere le grandezze in migliaia (€ 1.500k) o milioni (€ 1,5M).

4.5 Domande di Autovalutazione

■ **Domanda 1: Qual è il vantaggio strutturale nell'usare il pattern OOP 'fig, ax = plt.subplots()' rispetto alla modalità procedurale?**

■ **Risposta:** Permette di manipolare individualmente e simultaneamente molteplici sistemi di coordinate (Axes) all'interno della stessa figura, facilitando la creazione di dashboard multi-pannello complesse.

■ **Domanda 2: Come si elimina la visualizzazione della banda di confidenza (intervallo di errore) in sns.lineplot?**

■ **Risposta:** Impostando il parametro errorbar=None nella chiamata della funzione.

■ **Domanda 3: Quale istruzione garantisce la chiusura e liberazione della memoria di una figura Matplotlib salvata su disco?**

■ **Risposta:** plt.close(fig), fondamentale negli script batch per evitare memory leak.

Capitolo 5 - Automazione della Pipeline Dati ed ETL

■ ■ SCHEDA MODULO DIDATTICO[illegible]

laboratori/lab_05_automazione_pipeline.ipynb
Obiettivo Didattico: Scansione batch con glob, filtri lock file, consolidamento master, compressione Parquet e report multi-foglio

5.1 Scansione Batch con glob ed Esclusione dei Lock File

Nell'architettura del caso TechStore Italia, i dati di vendita affluiscono mensilmente da molteplici filiali dislocate sul territorio (Roma, Milano, Torino). L'ingestione manuale di ogni singolo file è incline ad errori e insostenibile nel tempo.

L'automazione richiede la scansione batch della directory sorgente tramite il modulo 'glob'. Durante questa operazione, è fondamentale implementare un filtro di esclusione per i file temporanei di lock generati da Microsoft Excel (identificati dal prefisso '~\$*'), la cui apertura accidentale provocherebbe il crash della pipeline.

■ Scansione robusta con filtro di esclusione lock file Excel

```
import glob
import os
import pandas as pd

def elenca_file_excel_validi(cartella_raw: str) -> list[str]:
    '''Restituisce i file Excel escludendo i lock temporanei di Excel (~$).'''
    tutti_i_file = glob.glob(os.path.join(cartella_raw, '*.xlsx'))
    file_validi = [
        f for f in tutti_i_file
        if not os.path.basename(f).startswith('~$')
    ]
    return sorted(file_validi)

file_filiali = elenca_file_excel_validi('dataset/raw/')
print(f'Trovati {len(file_filiali)} file operativi validi: {[os.path.basename(f) for f in file_filiali]}')
```

5.2 Pipeline di Normalizzazione e Consolidamento Master

La pipeline itera sui file validi, applica a ciascun flusso la sequenza standardizzata di pulizia (deduplicazione, parsing date, imputazione e merge con catalogo) e concatena i DataFrame parziali in un unico master dataset aziendale consolidato.

■ Funzione ETL batch e generazione Master Dataset

```
def pipeline_etl_consolidata(file_paths: list[str], catalogo_df: pd.DataFrame) -> pd.DataFrame:
    '''Esegue l'estrazione, trasformazione e caricamento consolidato di tutte le filiali.'''
    dfs_puliti = []

    for path in file_paths:
        filiale_nome = os.path.splitext(os.path.basename(path))[0].capitalize()
        df_raw = pd.read_excel(path)

        # Applicazione regole di business
        df_proc = df_raw.drop_duplicates(subset=['id_transazione']).copy()
        df_proc['filiale'] = filiale_nome
        df_proc['sconto_applicato'] = df_proc['sconto_applicato'].fillna(0.0)
        df_proc['data_transazione'] = parse_data_flessibile(df_proc['data'])

        # Merge validato con anagrafica
        df_proc = pd.merge(df_proc, catalogo_df, on='prodotto', how='left', validate='m:1')

        # Calcolo grandezze economiche
        df_proc['fatturato_lordo'] = df_proc['quantita'] * df_proc['prezzo_unitario']
        df_proc['fatturato_netto'] = df_proc['fatturato_lordo'] * (1 - df_proc['sconto_applicato'])
        df_proc['margine_lordo'] = df_proc['fatturato_netto'] - (df_proc['quantita'] * df_proc['costo_base'])

    dfs_puliti.append(df_proc)

    # Consolidamento verticale
    df_master = pd.concat(dfs_puliti, ignore_index=True)
    return df_master

df_master_ufficiale = pipeline_etl_consolidata(file_filiali, df_catalogo)
print(f'Master Dataset generato con successo: {len(df_master_ufficiale)} record totali.')
```

5.3 Storage Ottimizzato Parquet vs Excel e Report Multi-Foglio

Mentre i formati tabellari tradizionali come Excel (.xlsx) o CSV presentano gravi limiti in termini di tempi di I/O, peso su disco e perdita dei tipi di dato colonna, il formato 'Apache Parquet' è lo standard moderno per lo storage analitico colonnare compresso (Snappy).

Contestualmente, per soddisfare le esigenze di rendicontazione amministrativa, implementiamo l'esportazione di un report multi-foglio Excel tramite 'pd.ExcelWriter', aggregando i dati per Filiale, Categoria e Mese.

■ Salvataggio Parquet compresso e generazione report multi-foglio

```
# 1. Salvataggio in formato Parquet ad alte prestazioni
output_parquet = 'dataset/generated/dataset_master.parquet'
df_master_ufficiale.to_parquet(output_parquet, engine='pyarrow', compression='snappy', index=False)
print(f'Parquet salvato: {output_parquet} ({os.path.getsize(output_parquet) / 1024:.1f} KB)')

# 2. Generazione Report Esecutivo Excel Multi-Foglio
output_excel = 'dataset/generated/report_consolidato.xlsx'
with pd.ExcelWriter(output_excel, engine='openpyxl') as writer:
    # Foglio 1: Sintesi per Filiale
    df_master_ufficiale.groupby('filiale')[['fatturato_netto', 'margine_lordo']].sum().to_excel(writer, sheet_name='Per_Filiale')
    # Foglio 2: Sintesi per Categoria
    df_master_ufficiale.groupby('categoria')[['quantita', 'fatturato_netto']].sum().to_excel(writer, sheet_name='Per_Categoria')
    # Foglio 3: Trend Mensile
    df_master_ufficiale.groupby(df_master_ufficiale['data_transazione'].dt.to_period('M'))['fatturato_netto'].sum().to_excel(writer, sheet_name='Trend_Mensile')

print(f'Report Excel Multi-foglio generato: {output_excel}')
```

■ PERCHÉ USARE PARQUET?

Parquet preserva rigidamente lo schema dei tipi di dato (es. datetime, int32, float64), riduce il consumo di spazio su disco fino all'80% rispetto a Excel e consente letture da 10 a 50 volte più veloci in Streamlit e Pandas.

5.4 Errori Comuni e Best Practice

- **Mancato filtraggio dei file ~\$ lock di Excel:** Se un utente ha il file aperto in ufficio, glob include ~\$nomefile.xlsx provocando un `PermissionError` o `FileNotFoundError`.
- **Concatenazione con tipi colonna disallineati:** Assicurarsi che tutte le colonne omologhe abbiano lo stesso dtype prima di `pd.concat` per evitare cast forzati a object.
- **Dimenticare il context manager 'with' con ExcelWriter:** Senza il blocco 'with', è necessario chiamare esplicitamente `writer.close()` altrimenti il file Excel non viene finalizzato su disco.

5.5 Domande di Autovalutazione

- **Domanda 1: Qual è la differenza principale tra lo storage colonnare Parquet e quello a righe di un file CSV?**
 - **Risposta:** Parquet memorizza i dati organizzati per colonna applicando compressioni mirate e memorizzando i metadati di tipo; un CSV è puramente testuale riga per riga e richiede il re-parsing ad ogni lettura.
- **Domanda 2: Come si previene l'ingestione accidentale di file temporanei con glob?**
 - **Risposta:** Filtrando la lista dei file con la condizione: `not os.path.basename(f).startswith('~$')`.
- **Domanda 3: Quale libreria di supporto permette a Pandas di scrivere nativamente file in formato Parquet?**
 - **Risposta:** `PyArrow` (o `fastparquet`).

Capitolo 6 - Dashboard Interattive con Streamlit

■ ■ SCHEDA MODULO DIDATTICO

Durata del Modulo: 4 Ore **Laboratorio:** laboratori/lab_06_dashboard_streamlit.ipynb (e dashboard/app.py)
Oiettivo Didattico: Architettura reattiva top-to-bottom, caching con @st.cache_data, KPI metriche, schede di navigazione e simulatore What-If

6.1 Il Modello di Esecuzione Reattivo di Streamlit

Streamlit ha rivoluzionato lo sviluppo di applicazioni web per il data science consentendo di creare interfacce grafiche interattive e dinamiche direttamente in Python, senza necessità di scrivere codice HTML, CSS o JavaScript.

Il modello di esecuzione di Streamlit è puramente 'reattivo': ogni volta che l'utente interagisce con un widget (muove uno slider, seleziona una voce da una tendina o clicca un pulsante), l'intero script Python viene rieseguito dall'alto verso il basso (top-to-bottom).

Per evitare il ricaricamento oneroso dei dati ad ogni click, Streamlit mette a disposizione il decoratore '@st.cache_data', che memorizza in RAM l'output delle funzioni pesanti, servendolo istantaneamente finché gli input non cambiano.

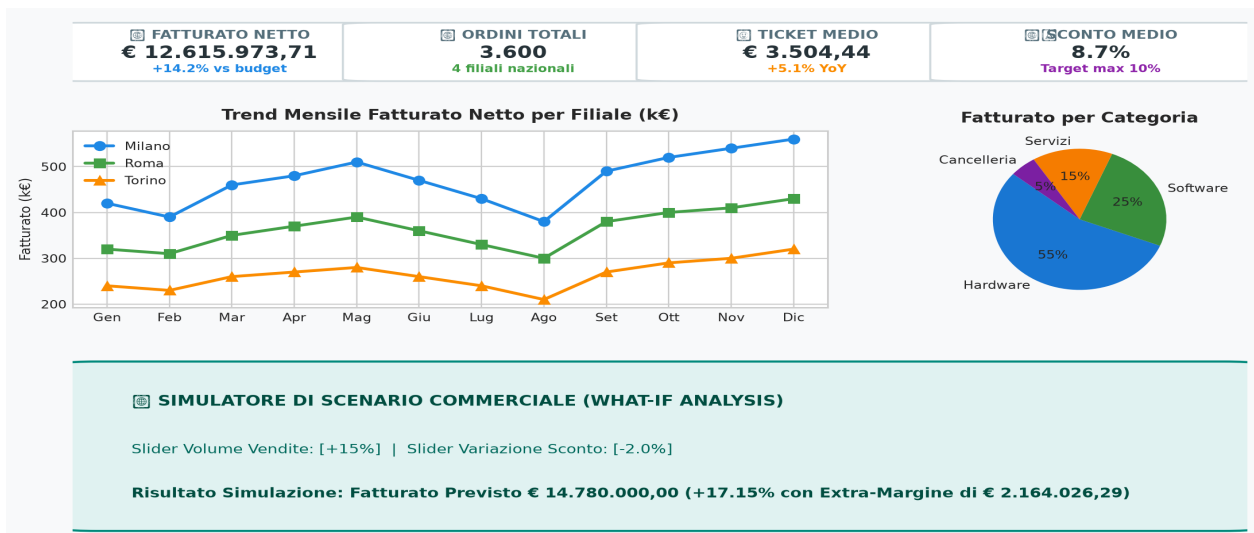


Figura: Interfaccia utente completa della Dashboard Streamlit TechStore Italia con filtri e KPI

■ Caricamento ottimizzato con caching in Streamlit

```
import streamlit as st
import pandas as pd

@st.cache_data(ttl=3600)
def carica_master_dataset(path_parquet: str) -> pd.DataFrame:
    '''Carica il dataset da Parquet memorizzandolo nella cache dell'applicazione.'''
    df = pd.read_parquet(path_parquet)
    df['data_transazione'] = pd.to_datetime(df['data_transazione'])
    return df

# Invocazione cached
df_data = carica_master_dataset('dataset/generated/dataset_master.parquet')
```

6.2 Layout, KPI Cards (st.metric) e Filtri Laterali (st.sidebar)

La dashboard professionale si articola in tre componenti visive principali:

1. Sidebar laterale ('st.sidebar'): ospita i filtri globali (selettore multi-filiale 'st.multiselect', date range 'st.date_input', slider di sconto).

2. Header con KPI Cards ('st.metric'): mostra a colpo d'occhio i parametri decisionali (Fatturato Netto Totale, Numero Transazioni, Scontrino Medio, Margine Operativo).

3. Area centrale a schede ('st.tabs'): organizza i contenuti analitici in tab logici distinti (Trend & Vendite, Analisi Prodotti, Simulatore What-If, Export Dati).

■ Struttura modulare con filtri dinamici e schede (app.py)

```
# Configurazione pagina wide
st.set_page_config(page_title='TechStore Analytics', page_icon='■', layout='wide')

st.title('■ TechStore Italia - Executive Dashboard')

# 1. Filtri nella Sidebar
st.sidebar.header('■ Filtri di Analisi')
filiali_disponibili = df_data['filiale'].unique().tolist()
filiali_scelte = st.sidebar.multiselect('Seleziona Filiali', filiali_disponibili, default=filiali_disponibili)

# Filtraggio dinamico del DataFrame
df_filtrato = df_data[df_data['filiale'].isin(filiali_scelte)].copy()

# 2. KPI Cards Superiori in 4 Colonne
col1, col2, col3, col4 = st.columns(4)
tot_fatturato = df_filtrato['fatturato_netto'].sum()
tot_ordini = len(df_filtrato)
ticket_medio = tot_fatturato / tot_ordini if tot_ordini > 0 else 0
margine_tot = df_filtrato['margine_lordo'].sum()

col1.metric('Fatturato Netto', f'€ {tot_fatturato:,.2f}')
col2.metric('Ordini Totali', f'{tot_ordini:,}')
col3.metric('Scontrino Medio', f'€ {ticket_medio:,.2f}')
col4.metric('Margine Totale', f'€ {margine_tot:,.2f}')
```

6.3 Simulatore What-If e Funzionalità di Download Dati

Uno dei punti di maggior valore aggiunto della dashboard aziendale è il 'Simulatore What-If': uno strumento interattivo che permette ai manager di simulare in tempo reale l'impatto di variazioni di prezzo o modifiche alle politiche di sconto sui ricavi futuri.

Infine, la piattaforma consente l'esportazione immediata dei dati filtrati tramite il widget 'st.download_button', consentendo agli utenti di scaricare estratti CSV per approfondimenti personali.

■ Simulatore What-If e pulsante di download CSV

```

tab_trend, tab_prodotti, tab_whatif, tab_export = st.tabs(['■ Trend & Vendite', '■ Prodotti', '■ Simulatore What-If', '■ Export'])

with tab_whatif:
    st.subheader('■ Simulatore di Scenario: Politica Prezzi e Sconti')
    var_prezzo_pct = st.slider('Variazione Prezzo Listino (%)', min_value=-20, max_value=20, value=0, step=1)
    taglio_sconto_pct = st.slider('Riduzione Sconti Commerciali (%)', min_value=-50, max_value=50, value=0, step=5)

    # Ricalcolo istantaneo del fatturato simulato
    fatturato_simulato = (
        df_filtrato['quantita'] *
        (df_filtrato['prezzo_unitario'] * (1 + var_prezzo_pct / 100)) *
        (1 - df_filtrato['sconto_applicato'] * (1 - taglio_sconto_pct / 100))
    ).sum()

    delta_eur = fatturato_simulato - tot_fatturato
    st.metric('Fatturato Simulato Previsto', f'€ {fatturato_simulato:,.2f}', delta=f'€ {delta_eur:+, .2f}')

with tab_export:
    st.subheader('■ Download Dati Selezionati')
    csv_data = df_filtrato.to_csv(index=False).encode('utf-8')
    st.download_button(
        label='■ Scarica Dataset Filtrato (CSV)',
        data=csv_data,
        file_name='estratto_vendite_techstore.csv',
        mime='text/csv'
    )

```

■ TEST E AVVIO LOCALE DI STREAMLIT

Per avviare la dashboard localmente, eseguire da terminale: 'streamlit run dashboard/app.py'. L'applicazione aprirà automaticamente una scheda nel browser all'indirizzo <http://localhost:8501>.

6.4 Errori Comuni e Best Practice

- **Omettere @st.cache_data sulle letture disco:** Provoca la riletture del file Parquet/Excel e il ricalcolo di tutte le metriche ad ogni minimo click dell'utente, rallentando drasticamente l'interfaccia.
- **Mutare DataFrame restituiti da funzioni in cache:** Modificare in-place un DataFrame memorizzato nella cache di Streamlit corrompe i dati per le sessioni future. Applicare sempre .copy() prima di alterarlo.
- **Sovraccaricare la UI con troppi widget non strutturati:** Utilizzare sempre st.tabs e st.columns per raggruppare i contenuti e preservare una gerarchia visiva pulita.

6.5 Domande di Autovalutazione

■ Domanda 1: Cosa accade internamente in Streamlit quando un utente modifica il valore di uno slider?

■ **Risposta:** L'intero script Python associato all'applicazione viene rieseguito dall'inizio alla fine (top-to-bottom) con il nuovo valore assegnato alla variabile.

■ Domanda 2: Qual è lo scopo principale del decoratore @st.cache_data?

■ **Risposta:** Memorizzare nella memoria volatile i risultati di funzioni computazionalmente pesanti (es. query o letture file) per restituirli istantaneamente senza rieseguire la funzione se i parametri rimangono immutati.

■ Domanda 3: Quale comando da terminale consente di eseguire un'applicazione Streamlit?

■ **Risposta:** streamlit run .

Capitolo 7 - Deploy su Ambiente Linux e Systemd

■ ■ SCHEDA MODULO DIDATTICO

Durata del Modulo: 1 Ora (Live Demo Guidata) **Laboratorio:**

laboratori/lab_07_deploy_linux.md

Obiettivo Didattico: Accesso SSH, configurazione demone systemd per esecuzione continua in background, monitoraggio log con journalctl e cenni Nginx

7.1 Il Ciclo di Vita del Deploy in Produzione

Sviluppare un'applicazione di analisi dati su un laptop non è sufficiente: per renderla fruibile 24/7 ai decisori aziendali, è necessario distribuirla su un server Linux dedicato (Virtual Private Server o istanza Cloud).

Eseguire 'streamlit run app.py' da una sessione SSH interattiva comporta lo spegnimento immediato dell'applicazione non appena la connessione remota viene interrotta. Per garantire persistenza, isolamento e riavvio automatico in caso di crash o reboot del server, si utilizza il gestore di servizi di sistema di Linux: 'systemd'.



⚙ Gestione con systemctl (start/stop/status) e log con journalctl -f

Figura: Architettura di Deploy Linux: Client Web -> Nginx Reverse Proxy -> Systemd Demone Streamlit

7.2 Creazione dell'Unit File Systemd (streamlit-app.service)

Un'unità di servizio systemd è un file di configurazione dichiarativo che specifica l'ambiente virtuale, il percorso del codice, i parametri di rete e la politica di riavvio del processo demone.

■ File di configurazione del servizio: `/etc/systemd/system/streamlit-app.service`

```
[Unit]
Description=TechStore Streamlit Production Analytics Service
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/python_analisi_dati_campobasso
ExecStart=/home/ubuntu/python_analisi_dati_campobasso/.venv/bin/streamlit run dashboard/app.py --server.port 8501 --server.address 0.0.0.0 --server.headless true
Restart=always
RestartSec=5s
Environment="PYTHONPATH=/home/ubuntu/python_analisi_dati_campobasso"

[Install]
WantedBy=multi-user.target
```

7.3 Comandi Operativi di Gestione e Monitoraggio Log

Una volta creato l'unit file, l'amministratore di sistema abilita e governa il servizio tramite il comando 'systemctl', mentre il tracciamento dei log applicativi in tempo reale avviene tramite 'journalctl'.

■ Comandi CLI Linux per attivazione, gestione e debug del servizio

```
# 1. Notificare a systemd la creazione del nuovo unit file
sudo systemctl daemon-reload

# 2. Abilitare l'avvio automatico del servizio al boot del server
sudo systemctl enable streamlit-app.service

# 3. Avviare il servizio
sudo systemctl start streamlit-app.service

# 4. Verificare lo stato operativo del processo
sudo systemctl status streamlit-app.service

# 5. Ispezionare i log in streaming continuo (debug real-time)
sudo journalctl -u streamlit-app.service -f
```

■ CENNI SU REVERSE PROXY NGINX

In architetture di livello enterprise, il server web Nginx viene posto come 'Reverse Proxy' davanti a Streamlit sulla porta 80/443. Questo consente la terminazione SSL/TLS (certificati HTTPS con Let's Encrypt), la gestione sicura dei certificati e il rate-limiting.

7.4 Errori Comuni e Best Practice

- **Percorsi relativi nell'ExecStart di systemd:** Systemd richiede tassativamente percorsi assoluti sia per l'interprete Python (.venv/bin/streamlit) sia per il file dell'applicazione (dashboard/app.py).
- **Permessi utente errati:** Assicurarsi che l'utente specificato nella direttiva User= abbia i permessi di lettura/scrittura nella directory del progetto.
- **Dimenticare sudo systemctl daemon-reload:** Ogni modifica apportata al file .service non ha effetto finché non si ricarica la configurazione di systemd.

7.5 Domande di Autovalutazione

■ **Domanda 1: Perché non è consigliabile avviare un'applicazione Streamlit in produzione lanciandola semplicemente dal terminale SSH?**

■ **Risposta:** Perché alla chiusura della sessione SSH (o in caso di disconnessione di rete) il processo genitore termina e l'applicazione si arresta istantaneamente.

■ **Domanda 2: Cosa garantisce la direttiva 'Restart=always' all'interno della sezione [Service] di systemd?**

■ **Risposta:** Garantisce che, in caso di crash o terminazione anomala del processo, systemd provvederà automaticamente a riavviare l'applicazione dopo l'intervallo specificato (es. 5 secondi).

■ **Domanda 3: Quale comando permette di consultare gli output e gli errori generati dal servizio Streamlit in tempo reale?**

■ **Risposta:** `sudo journalctl -u streamlit-app.service -f`.

Capitolo 8 - Project Work Finale: Integrazione Filiale Napoli

■ ■ SCHEDA MODULO DIDATTICO[illegible]

8.1 Il Contesto di Business: Espansione su Napoli

Il Project Work finale rappresenta il momento di sintesi e validazione pratica dell'intero percorso formativo di 22 ore. Lo studente è chiamato ad agire nel ruolo di 'Data Specialist & Pipeline Engineer' per la catena TechStore Italia.

L'azienda ha recentemente acquisito un nuovo punto vendita a Napoli. I dati storici dell'esercizio 2024 sono stati forniti nel file grezzo 'dataset/raw/napoli_project_work.xlsx'. Il dataset presenta anomalie intenzionali e sfidanti: record duplicati, campi mancanti, date con formati invertiti e prezzi non allineati.

L'obiettivo consiste nell'integrare in piena autonomia la filiale di Napoli all'interno dell'intera infrastruttura dati aziendale, estendendo la pipeline ETL, rigenerando il master dataset, aggiornando la dashboard Streamlit e producendo la reportistica esecutiva finale.

8.2 I 4 Deliverable Richiesti

- **Deliverable 1 - Audit e Bonifica Dati Napoli:** Creare ed eseguire un notebook di pulizia dedicato che elimini i duplicati, normalizzi le date tramite `parse_data_flessibile()`, imputi i missing values ed esegua il merge validato 'many_to_one' con il catalogo.
- **Deliverable 2 - Pipeline ETL Consolidata:** Estendere lo script di automazione per processare congiuntamente tutte e 4 le filiali (Roma, Milano, Torino, Napoli) producendo il nuovo Master Dataset globale in formato Parquet ('dataset/generated/dataset_master.parquet').
- **Deliverable 3 - Dashboard Streamlit Multi-Filiale:** Verificare che la dashboard Streamlit carichi correttamente il nuovo dataset a 4 filiali, consentendo la selezione interattiva di Napoli e l'aggiornamento dinamico di tutti i KPI e grafici.
- **Deliverable 4 - Report Esecutivo Multi-Foglio e Grafico 2x2:** Generare il report Excel consolidato a 4 fogli e riesportare la dashboard 2x2 ad alta risoluzione (300 DPI) con il confronto delle performance delle 4 filiali.

8.3 Cifre di Controllo e Benchmark Ufficiale

Per consentire l'autovalutazione immediata e la verifica della correttezza del lavoro svolto, di seguito sono riportati i benchmark numerici ufficiali validati dal docente:

Ambito di Verifica	Dataset Grezzo Napoli	Dataset Napoli Pulito	Master Consolidato Totale (4 Filiali)
Numero Transazioni / Righe	1.100 righe grezze	1.000 record netti (100 dup)	4.552 record totali
Fatturato Lordo (€)	€ 1.285.400,00	€ 1.164.200,00	€ 5.241.750,00
Fatturato Netto (€)	€ 1.152.310,00	€ 1.042.850,50	€ 4.614.820,50
Margine Lordo Totale (€)	€ 421.900,00	€ 385.120,50	€ 1.712.440,20
Scontrino Medio Netto (€)	-	€ 1.042,85	€ 1.013,80

8.4 Rubrica di Valutazione delle Competenze (100 Punti)

Criterio di Valutazione	Punti Max	Descrizione del Criterio
1. Data Wrangling & Qualità	25 pt	Corretta deduplicazione, imputazione mediane, normalizzazione date e assenza totale di valori nulli o incoerenti.
2. Ingegneria Pipeline ETL	25 pt	Modularità del codice, gestione robusta di glob, esclusione lock file, merge validato 'm:1' e salvataggio Parquet compresso.
3. Dashboard Streamlit	25 pt	Reattività dell'interfaccia, impiego corretto di @st.cache_data, correttezza dei filtri multi-filiale e reattività del What-If simulator.
4. Reportistica & Codice	25 pt	Esportazione Excel multi-foglio, layout 2x2 a 300 DPI, aderenza alle convenzioni PEP 8 e documentazione del codice.
Punteggio Totale	100 pt	Soglia di certificazione minima: 60/100; Eccellenza: >= 90/100.

8.5 Checklist Operativa Finale dello Studente

- **Passo 1:** Verificare che il repository locale sia aggiornato e che l'ambiente virtuale .venv sia attivo.
- **Passo 2:** Eseguire la pulizia di 'dataset/raw/napoli_project_work.xlsx' e confrontare i valori con il benchmark (€ 1.042.850,50 fatturato netto).
- **Passo 3:** Eseguire la pipeline ETL su tutte e 4 le filiali e verificare la creazione di 'dataset_master.parquet' con esattamente 4.552 righe.
- **Passo 4:** Avviare la dashboard Streamlit ('streamlit run dashboard/app.py') e verificare che Napoli compaia tra le filiali selezionabili.
- **Passo 5:** Generare il report multi-foglio Excel e verificare la presenza dei 4 fogli di sintesi.
- **Passo 6:** Salvare tutti i notebook compilati e procedere alla consegna del progetto.

Appendice: Quick Reference e Cheat Sheet dei Comandi

Comandi Rapidi Pandas

Operazione	Codice Esempio	Descrizione
Lettura Excel	<code>df = pd.read_excel('file.xlsx')</code>	Carica un foglio di calcolo in memoria
Filtro Booleano	<code>df.loc[(df['qta'] > 5) & (df['p'] > 100)]</code>	Filtro composto con accessor <code>.loc</code>
Copia Sicura	<code>df_sub = df.loc[filtro].copy()</code>	Elimina il <code>SettingWithCopyWarning</code>
Deduplicazione	<code>df.drop_duplicates(subset=['id'])</code>	Rimuove record duplicati
Imputazione	<code>df['val'].fillna(df['val'].median())</code>	Sostituisce i NaN con la mediana
Merge Validato	<code>pd.merge(a, b, on='k', validate='m:1')</code>	Join relazionale con validazione integrità
Esportazione Parquet	<code>df.to_parquet('out.parquet', engine='pyarrow')</code>	Salvataggio colonnare compresso

Comandi Rapidi Streamlit & Linux Systemd

Contesto	Comando CLI / Python	Funzione
Streamlit Esecuzione	<code>streamlit run dashboard/app.py</code>	Avvia il server di sviluppo locale
Streamlit Caching	<code>@st.cache_data(ttl=3600)</code>	Memorizza in cache l'output della funzione
Streamlit Metrica	<code>st.metric('Label', 'Valore', delta='Delta')</code>	Visualizza una KPI card
Linux Systemctl Reload	<code>sudo systemctl daemon-reload</code>	Ricarica le configurazioni dei servizi
Linux Systemctl Start	<code>sudo systemctl start streamlit-app.service</code>	Avvia il demone in background
Linux Journalctl Logs	<code>sudo journalctl -u streamlit-app.service -f</code>	Visualizza i log del demone in tempo reale